

Trabalho I - Máquina de Estados

1. Visão geral do cenário

O cenário simula um Board Game Café. O ambiente é composto por três agentes: dois funcionários com funções distintas (um barista e um monitor de jogos) e um cliente. O ecossistema funciona em um ciclo de dependência, onde as necessidades fisiológicas e mecânicas do cliente (sede e troca de jogos) disparam ações (pedidos de café e solicitações de ajuda) que interrompem as funções de limpeza e descanso dos funcionários, gerando transições de estado dinâmicas e comunicativas entre os três agentes.

2. Visão geral de cada agente

2.1 Agente - 1

Funcionário responsável pelo balcão. Quando não há nenhum pedido, ele alterna entre limpar o bar e descansar. Porém, se receber um pedido, ele interrompe sua ação atual para preparar a bebida.

2.2 Agente - 2

Funcionário responsável pela organização do salão. Quando não há nenhum pedido de ajuda, ele alterna entre arrumar as prateleiras de jogos e descansar. Porém, se receber um pedido de ajuda, ele interrompe sua ação atual para ajudar o cliente

2.3 Agente - 3

Cliente responsável por manter os funcionários ativos. Ele passa a maior parte do tempo jogando, mas ao ficar cansado, ele altera seu estado para solicitar um café, voltando ao jogo apenas após consumi-lo. Quando a partida termina, ele pega um novo jogo e emite um pedido de ajuda, esperando até que o funcionário do salão lhe explique as regras.

3. Diagrama de estados por agente

3.1 Agente - 1

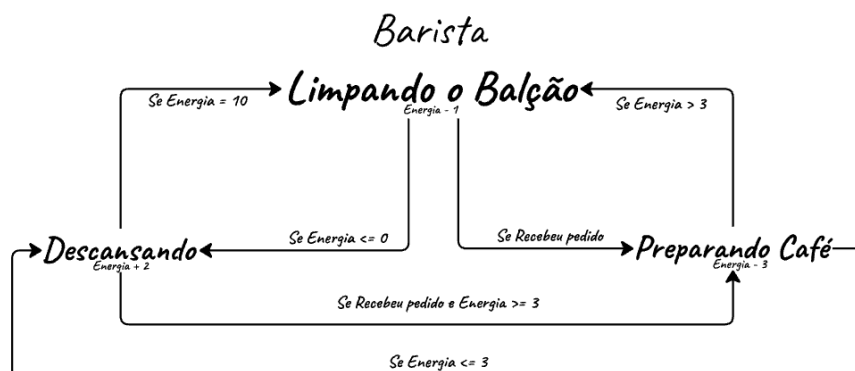


Figura 1 - Diagrama do agente 1 (Barista).

3.2 Agente - 2

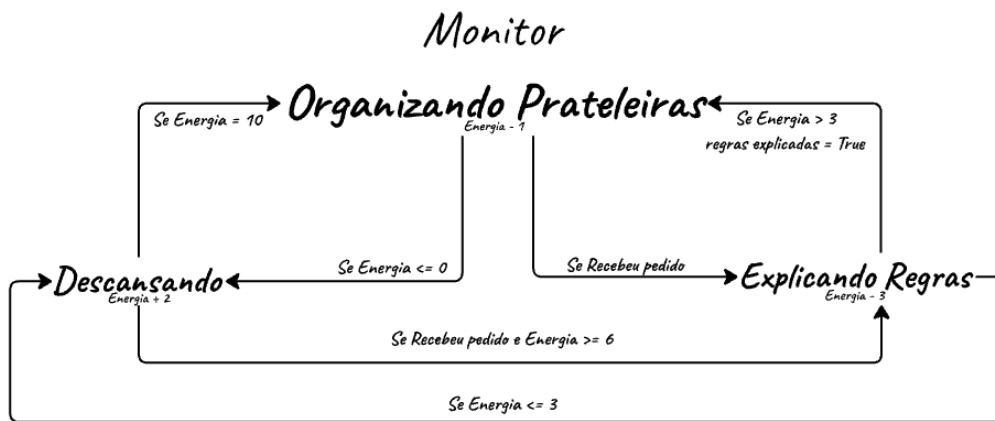


Figura 2 - Diagrama do agente 2 (Monitor).

3.3 Agente - 3

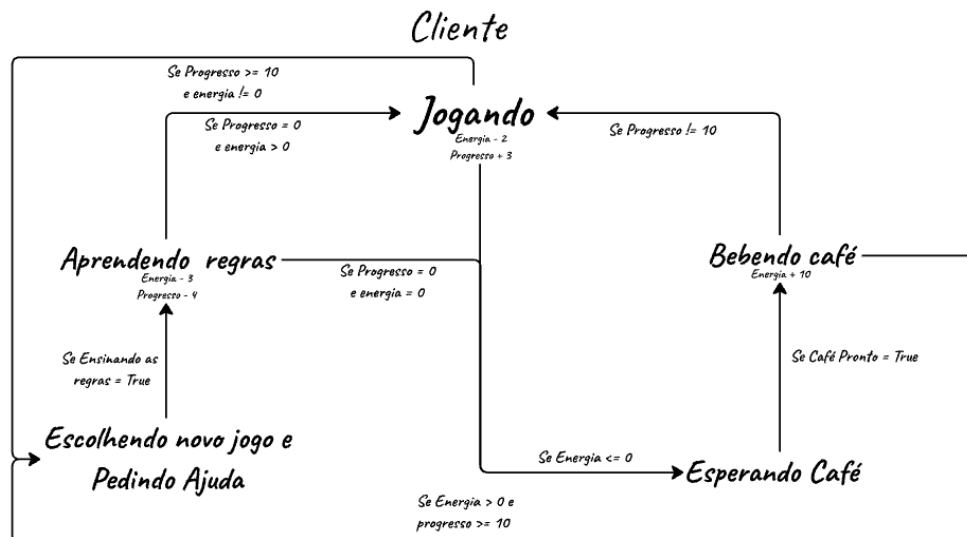


Figura 3 - Diagrama do agente 3 (Cliente).

4. Tabela de regras por agente

4.1 Agente - 1

Estado	Ação de Entrada (enter)	Ação de Execução (execute)	Ação de Saída (leave)	Transição
LimpandoBalcão	Exibe mensagem de início da limpeza.	Perde 1 de energia por rodada. Verifica se existe pedido de café. Exibe mensagem de limpando balcão.	Exibe mensagem de interrupção ou fim da limpeza.	Se houver pedido e energia suficiente para preparar o café, vai para PreparandoCafe. Se a energia chegar a 0, vai para DescansandoBarista.
PreparandoCafé	Exibe mensagem de preparo do café.	Perde 3 de energia. Ao terminar, define cafePronto = true. Exibe mensagem de fazendo café	Exibe mensagem de conclusão do pedido.	Se a energia restante for maior que 3, vai para LimpandoBalcão. Caso contrário, vai para DescansandoBarista

Descansando	Exibe mensagem de descanso.	Recupera 2 de energia por rodada, sem ultrapassar 10. Verifica se existe pedido pendenteExibe mensagem de descansando	Exibe mensagem de retorno à atividade.	Se houver pedido pendente e energia suficiente para preparar café, vai para PreparandoCafe. Se a energia chegar a 10 sem pedido pendente, vai para LimpandoBalcao. Caso contrário, permanece descansando.
-------------	-----------------------------	---	--	---

4.2 Agente - 2

Estado	Ação de Entrada (enter)	Ação de Execução (execute)	Ação de Saída (leave)	Transição
Organizando Prateleiras	Exibe mensagem de início de organização.	Perde 1 de energia por rodada. Verifica se existe solicitação de ajuda pendente.Exibe mensagem de organizando prateleiras.	Exibe mensagem de interrupção ou fim da organização.	Se houver solicitação de ajuda e energia maior que 6, vai para ExplicandoRegras. Se a energia chegar a 0, vai para DescansandoMonitor.
Explicando Regras	Exibe mensagem de início da explicação..	Permanece ajudando o cliente e perde 3 de energia por rodada. Verifica se o cliente informou que aprendeu as regras.Exibe mensagem de ajudando cliente.	Exibe mensagem de conclusão do atendimento..	Quando ajudaConcluida == true, volta para OrganizandoPrateleiras se tiver energia maior que 3; caso contrário, vai para DescansandoMonitor. Enquanto o cliente não concluir o aprendizado, permanece nesse estado.
Descansando	Exibe mensagem de descanso.	Recupera 2 de energia por rodada, sem ultrapassar 10. Verifica se existe pedido pendenteExibe mensagem de descansando	Exibe mensagem de retorno à atividade.	Se houver pedido pendente e energia suficiente para preparar café, vai para PreparandoCafe. Se a energia chegar a 10 sem pedido pendente, vai para LimpandoBalcao. Caso contrário, permanece descansando.

4.3 Agente - 3

Estado	Ação de Entrada (enter)	Ação de Execução (execute)	Ação de Saída (leave)	Transição
Jogando	Exibe mensagem de início da partida.	Perde 2 de energia e ganha 3 de progresso por rodada. Exibe mensagem de jogando.	Exibe mensagem de fim da partida ou interrupção..	Se a energia chegar a 0, vai para EsperandoCafe. Se o progresso chega a 10 vai para EscolhendoJogo. Caso contrário, permanece jogando.
EsperandoCafe	Registra a solicitação de café e sinaliza o pedido ao barista.	Aguarda até cafePronto == trueExibe mensagem de esperando cafe	Exibe mensagem de recebimento de café.	Quando o café estiver pronto, vai para BebendoCafe. Caso contrário permanece esperando.
BebendoCafe	Exibe mensagem de consumo do café.	Recupera 10 de energia, e marca cafePronto == false	Exibe mensagem de consumo de café.	Se o progresso for menor que 10, volta para Jogando. Se o progresso for 10, vai para EscolhendoJogo.
EscolhendoJogo	Exibe mensagem de escolha de um novo jogo e registra a solicitação de ajuda.	Seleciona um novo jogo e aguarda o monitor iniciar o atendimentoExibe mensagem de esperando monitor	Exibe mensagem que o monitor chegou.	Quando o monitor inicia a explicação, vai para AprendendoRegras.
AprendendoRegras	Exibe mensagem de início de aprendizagem.	Perde 3 de energia e 4 de progresso por rodada. Continua aprendendo mesmo que a energia chegue em 0.Exibe mensagem de aprendendo	Informa o monitor que aprendeu as regras, definindo ajudaConcluida = true	Se o progresso for igual a 0 e a energia for 0, vai para EsperandoCafe. Caso a energia seja maior que 0 e o progresso igual a 0, volta para o Jogando

5. Descrição das variáveis/limiares por agente

5.1 Agente - 1

Variável: energia (Tipo: Inteiro int).

Intervalo e Valor Inicial: Limitada matematicamente entre 0 e 10. Inicia com o valor 10.

Limiares de Transição:

- Energia ≤ 0 (no estado LimpandoBalcao): Força a transição para DescansandoBarista por exaustão.
- Energia > 3 (no estado PreparandoCafe): Ao finalizar um café, se a energia restante for maior que 3, retorna para LimpandoBalcao. Caso contrário, vai para DescansandoBarista.
- Energia ≥ 3 (no estado DescansandoBarista): Limiar mínimo que permite ao Barista interromper o descanso caso receba um pedidoCafe.
- Energia ≥ 10 (no estado DescansandoBarista): Indica que a energia foi totalmente restaurada, engatilhando o retorno para LimpandoBalcao.

5.2 Agente - 2

Variável: energia (Tipo: Inteiro int).

Intervalo e Valor Inicial: Limitada matematicamente entre 0 e 10. Inicia com o valor 10.

Limiares de Transição:

- Energia ≤ 0 (no estado OrganizandoPrateleira): Força a transição para DescansandoMonitor.
- Energia > 3 (no estado ExplicandoRegras): Após o cliente sinalizar o fim da explicação, se a energia do Monitor for maior que 3, ele retorna para OrganizandoPrateleira. Caso seja menor ou igual a 3, vai para DescansandoMonitor.
- Energia ≥ 6 (no estado DescansandoMonitor): Limiar de energia exigido para que o Monitor aceite interromper a sua pausa para atender a um pedidoAjuda.
- Energia ≥ 10 (no estado DescansandoMonitor): Energia totalmente recuperada, retornando ao estado padrão de OrganizandoPrateleira.

5.3 Agente - 3

Variável 1: energia (Tipo: Inteiro int). Intervalo de 0 a 10. Inicia com o valor 10.

Variável 2: progresso (Tipo: Inteiro int). Intervalo de 0 a 10. Inicia com o valor 0.

Limiares de Transição:

- Energia ≤ 0 (no estado Jogando): O cliente fica exausto antes de terminar o jogo e transita para EsperandoCafe.
- progresso ≥ 10 (no estado Jogando): O cliente conclui a partida atual e transita para EscolhendoJogo.
- progresso < 10 (no estado BebendoCafe): Após restaurar a energia para 10, o cliente avalia o progresso; se for menor que 10, ele volta a Jogando para terminar a partida. Se for 10 ou mais, vai para EscolhendoJogo.
- Progresso ≤ 0 (no estado AprendendoRegras): Indica que o cliente assimilou as regras. Se energia ≤ 0 , ele vai para EsperandoCafe, caso contrário, inicia a partida em Jogando.

5.4 Variáveis globais

Tipo e Inicialização: Booleanas (boolean), todas iniciadas como false.

Limiares Lógicos (Flags):

- pedidoCafe e cafePronto: Controlam o fluxo de atendimento. O Cliente ativa o pedido, o Barista atende e ativa a flag de conclusão, e o Cliente consome o café (retornando a flag para false).

- pedidoAjuda, ensinandoRegras e ajudaConcluida: Sincronizam a interação onde o Cliente pede ajuda, o Monitor assume o estado de ensino bloqueando outras ações, e o Cliente sinaliza a conclusão para libertar o Monitor.

6. Estrutura do código

6.1 Agente - 1

Barista.java: Implementa Character. Gerencia a energia e a lógica de receber pedidos.

LimpandoBalcao.java: Estado padrão de limpeza. Herda de AbstractState<Barista>.

PreparandoCafe.java: Estado ativo de atendimento ao pedido. Herda de AbstractState<Barista>.

DescansandoBarista.java: Estado de recuperação de energia. Herda de AbstractState<Barista>.

6.2 Agente - 2

Monitor.java: Implementa Character. Gerencia a energia e a lógica de receber pedidos de ajuda.

OrganizandoPrateleiras.java: Estado padrão de arrumação. Herda de AbstractState<Monitor>.

ExplicandoRegras.java: Estado ativo de ensino ao cliente. Herda de AbstractState<Monitor>.

DescansandoMonitor.java: Estado de recuperação de energia. Herda de AbstractState<Monitor>.

6.3 Agente - 3

Cliente.java: Implementa Character. Gerencia cansaço, progresso e faz as requisições aos funcionários.

Jogando.java: Estado principal de progressão de jogo e desgaste físico. Herda de AbstractState<Cliente>.

EsperandoCafe.java: Estado passivo que aguarda a flag cafePronto == true. Herda de AbstractState<Cliente>.

BebendoCafe.java: Estado ativo que consome o café e recupera a energia do cliente. Herda de AbstractState<Cliente>.

EscolhendoJogo.java: Estado de transição que solicita a ajuda do monitor. Herda de AbstractState<Cliente>.

AprendendoRegras.java: Estado ativo de assimilação do novo jogo. Herda de AbstractState<Cliente>.

6.3 Core

Main.java: Responsável por instanciar os agentes (Barista, Monitor e Cliente), configurar os valores iniciais e gerir a máquina de estados através de um loop infinito. É aqui que o método update() de cada agente é chamado a cada ciclo e onde residem as flags estáticas de comunicação global.

Character.java: Interface base para todos os agentes (update, printStats, setState).

State.java: Interface genérica (State<C>) definindo enter(), execute(), leave().

AbstractState.java: Classe abstrata que implementa State<C> e gerencia a referência do agente.

7. Resultados esperados

===== Nova Rodada =====

Barista - Energia: 10, Estado: LimpandoBalcao

O monitor está sendo chamado por um cliente.

Monitor - Energia: 9, Estado: ExplicandoRegras

O monitor chegou para ensinar as regras do jogo.

Cliente - Energia: 10, Progresso: 0, Estado: AprendendoRegras